

Прогрессивные Веб приложения

Александр Менщиков,
к.т.н., доцент ИТМО

Содержание лекции

1. Некоторые браузерные API
2. WebDriver
3. Расширения браузера
4. Прогрессивные Веб приложения

Некоторые браузерные API

Доступные браузерные API

Браузерные API - это API, встроенные в браузер и предоставляющие встроенные функции, которые можно использовать в веб-приложении. Их также можно назвать веб-интерфейсами.

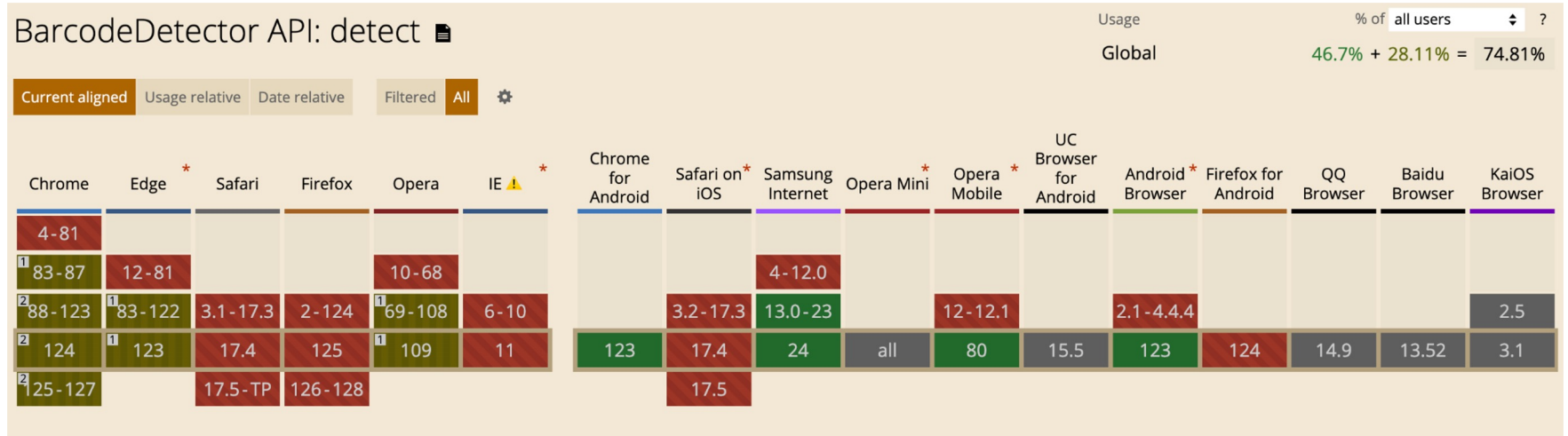
<https://developer.mozilla.org/en-US/docs/Web/API>

<https://caniuse.com/>

Barcode Detection API

https://developer.mozilla.org/en-US/docs/Web/API/Barcode_Detection_API

<https://caniuse.com/?search=BarcodeDetector>



Пример использования



ABC-1234

```
barcodeDetector = new BarcodeDetector({  
  formats: ["code_39", "qr_code"],  
});
```

```
barcodeDetector  
  .detect(document.getElementById(id))  
  .then((barcodes) => {  
    barcodes.forEach((barcode) => { textarea.innerHTML = barcode.rawValue });  
  })  
  .catch((err) => {  
    textarea.innerHTML = `Error: ${err}`;  
  });
```



Welcome to MDN docs

code 1

code 2

tainted canvas проблема

`file:///1-web-api/barcode.html`

Поскольку вы загружаете **изображение** из другого источника, доступ к фактическому содержимому изображения закрыт.

Вспоминаем, что `file://` – особый протокол с null origin.

выполняем для локальных тестов:

```
python -m http.server
```

EyeDropper API

<https://developer.mozilla.org/en-US/docs/Web/API/EyeDropper>

```
const eyeDropper = new EyeDropper();

eyeDropper
  .open()
  .then((result) => {
    resultElement.textContent = result.sRGBHex;
    resultElement.style.backgroundColor = result.sRGBHex;
  })
  .catch((e) => {
    resultElement.textContent = e;
  });
```

Open the eyedropper

#ede2ff

Fullscreen API

https://developer.mozilla.org/en-US/docs/Web/API/Fullscreen_API

```
document.documentElement.requestFullscreen();  
document.exitFullscreen();
```

Canvas API

https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API

```
<canvas id="canvas"></canvas>
```

```
<script>
```

```
    const canvas = document.getElementById("canvas");  
    const ctx = canvas.getContext("2d");
```

```
    ctx.fillStyle = "green";  
    ctx.fillRect(10, 10, 150, 100);  
    document.write(canvas.toDataURL());
```

```
</script>
```

data:image/png;base64,iVBORw0KGgoAAAANSUhEUgAAASwAAACWCAYAAABkW7XSAAAAA 10

WebDriver

WebDriver

WebDriver – это интерфейс удаленного управления, позволяющий осуществлять интроспекцию и контроль над пользовательскими агентами. Он предоставляет нейтральный для платформы и языка протокол для дистанционного управления веб-браузером.

Он предоставляет возможности для навигации по веб-страницам, пользовательского ввода, выполнения JavaScript и т.д.

<https://w3c.github.io/webdriver/#abstract>

Спецификация WebDriver

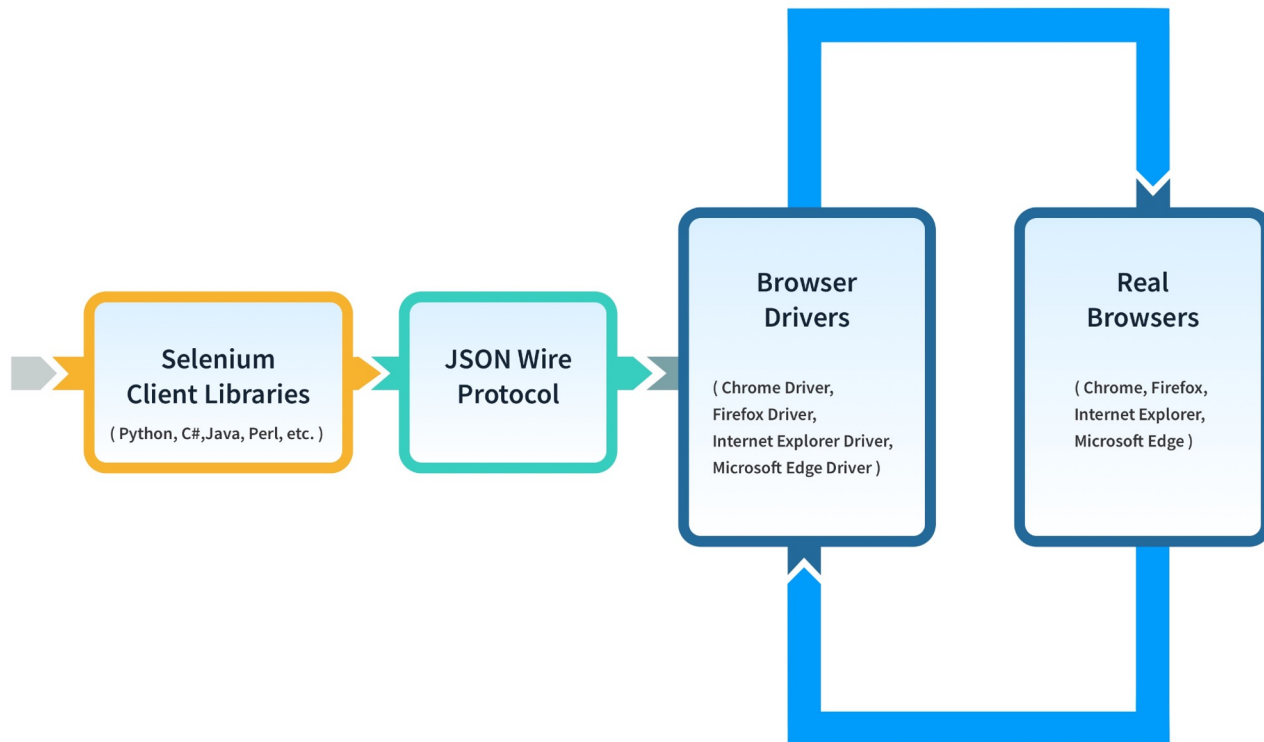
Эта спецификация основана на популярной платформе автоматизации браузера **Selenium WebDriver**

- HTTP compliant протокол
- JSON сериализация параметров и данных
- Использование API – <https://w3c.github.io/webdriver/#endpoints>

Method	URI Template	Command
POST	/session	New Session
DELETE	/session/{session id}	Delete Session
GET	/status	Status
GET	/session/{session id}/timeouts	Get Timeouts
POST	/session/{session id}/timeouts	Set Timeouts
POST	/session/{session id}/url	Navigate To
GET	/session/{session id}/url	Get Current URL
POST	/session/{session id}/back	Back
POST	/session/{session id}/forward	Forward
POST	/session/{session id}/refresh	Refresh

Упрощённая архитектура WebDriver

- Библиотека
- Протокол
- Драйвер
- Браузер



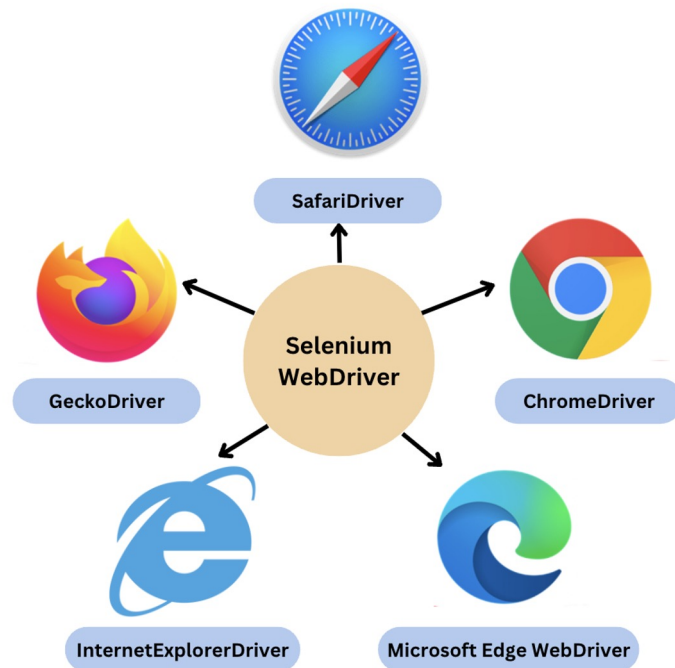
Реализации WebDriver

Chrome

<https://chromedriver.chromium.org/downloads>

<https://github.com/GoogleChromeLabs/chrome-for-testing?tab=readme-ov-file#find-the-latest-chrome-versions-across-channels>

ChromeDriver 124.0.6367.79

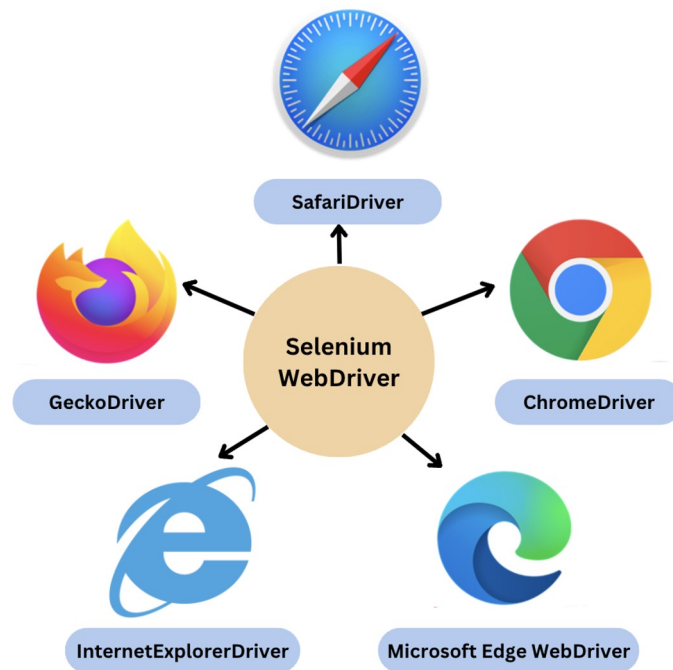


Реализации WebDriver

Firefox

<https://github.com/mozilla/geckodriver/releases>

geckodriver 0.34.0



Запускаем драйвер

1. Устанавливаем браузеры Chrome и Firefox
2. Загружаем исполняемый файл (например, **chromedriver** или **geckodriver**) версии совместимой с браузером
3. Запускаем

→ `./geckodriver`

```
1714128206688      geckodriver      INFO      Listening on 127.0.0.1:4444
```

→ `./chromedriver`

```
Starting ChromeDriver 124.0.6367.79
```

```
(b24d8cd7fe34716f461fee327262680a82fe93f2-refs/branch-heads/6367@{#955}) on port 9515
```

```
...
```

```
ChromeDriver was started successfully.
```

Отправляем запросы через cURL – создание сессии

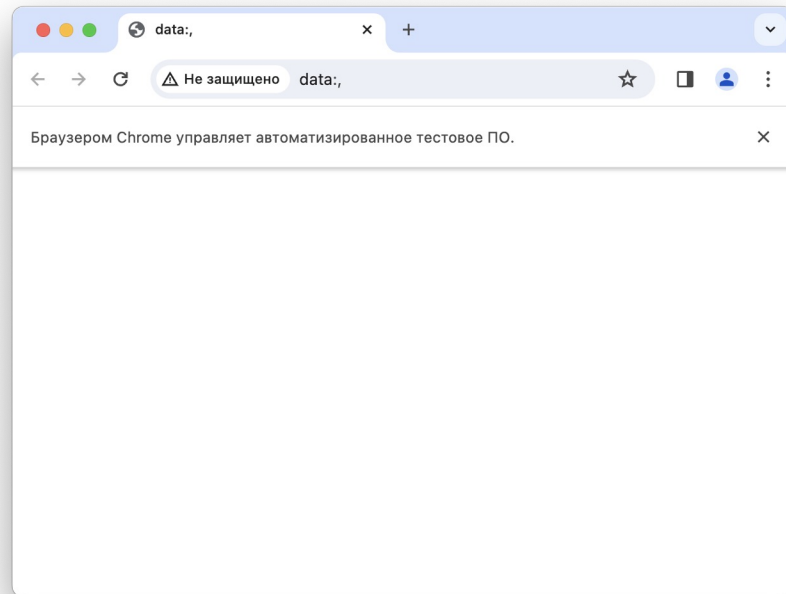
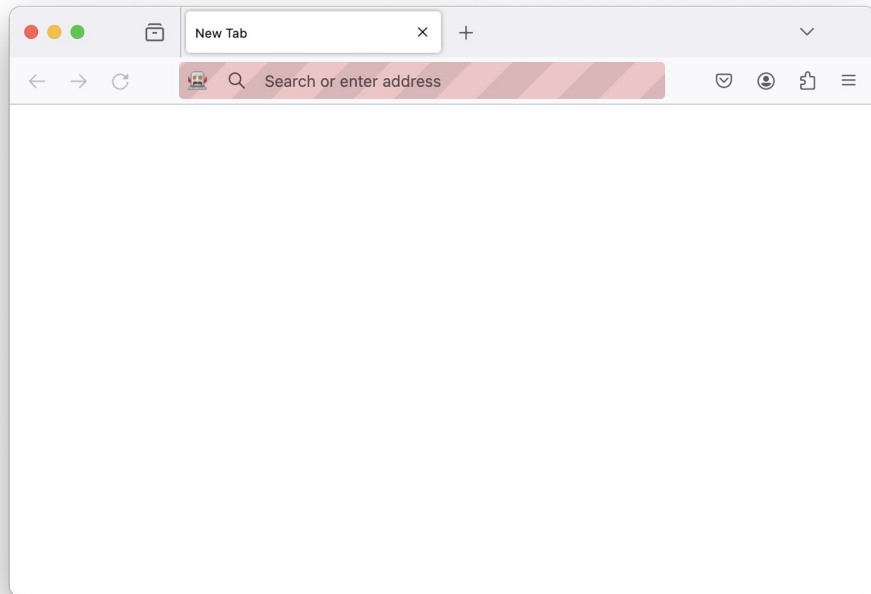
```
> curl -XPOST -H "Content-Type: application/json"  
http://localhost:4444/session -d  
'{"desiredCapabilities":{"browserName":"firefox"}}'
```

```
{"value":{"sessionId":"fd109dcc-03d6-4347-a238-a093db24cb8b","capabilities":{"acceptInsecureCerts":false,"browserName":"firefox","browserVersion":"124.0.2", ...
```

```
> curl -XPOST http://localhost:9515/session -d  
'{"desiredCapabilities":{"browserName":"chrome"}}'
```

```
{"sessionId":"30abc27f55ee77d49c695a22a799675d","status":0,"value":{"acceptInsecureCerts":false,"acceptSslCerts":false,"browserConnectionEnabled":false,"browserName":"chrome","chrome":{"chromedriverVersion":"124.0.6367.79 ...
```

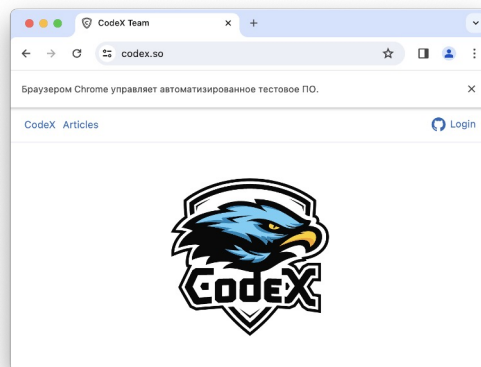
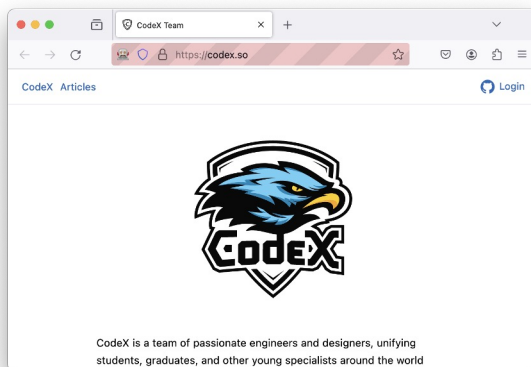
Запущены физические браузеры



Отправляем запросы через cURL – управление браузером

```
> curl -XPOST -H "Content-Type: application/json"  
http://localhost:4444/session/fd109dcc-03d6-4347-a238-  
a093db24cb8b/url -d '{"url":"https://codex.so"}'
```

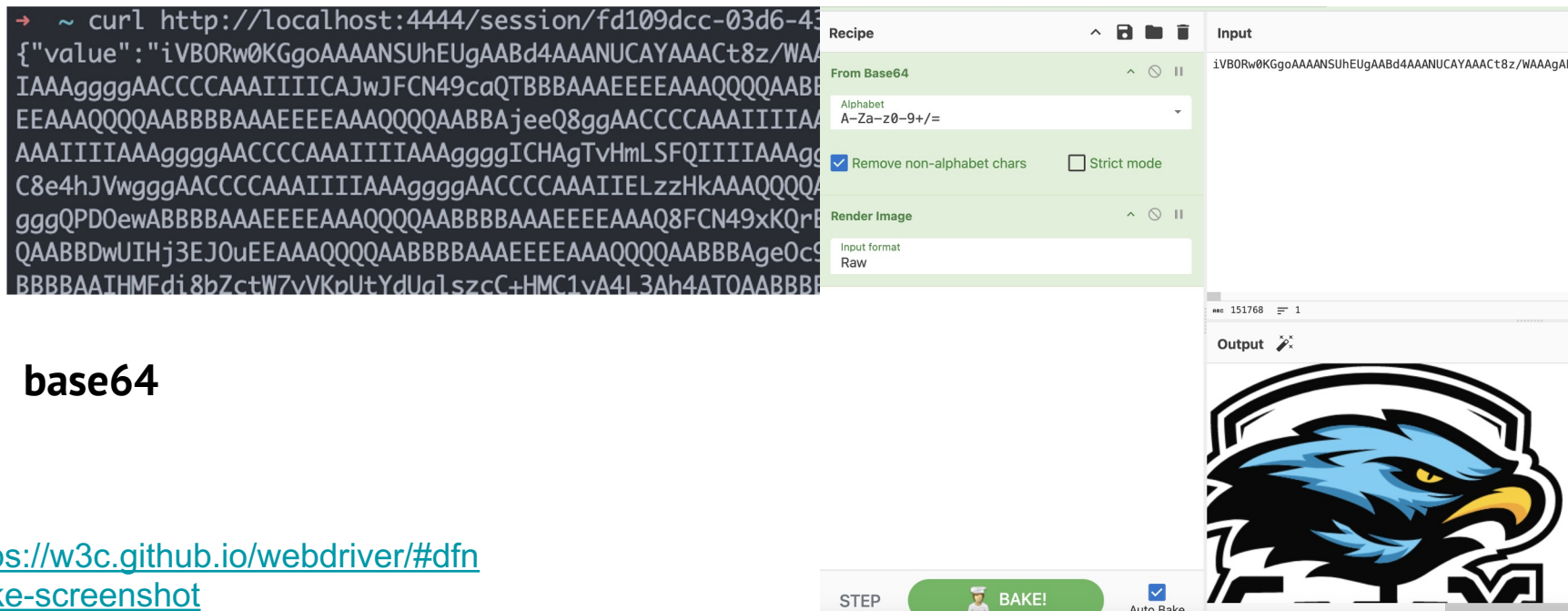
```
> curl -XPOST  
http://localhost:9515/session/30abc27f55ee77d49c695a22a799675d/url -  
d '{"url":"https://codex.so"}'
```



<https://w3c.github.io/webdriver/#dfn-navigate-to>

Пример со скриншотом

```
curl http://localhost:4444/session/fd109dcc-03d6-4347-a238-  
a093db24cb8b/screenshot
```

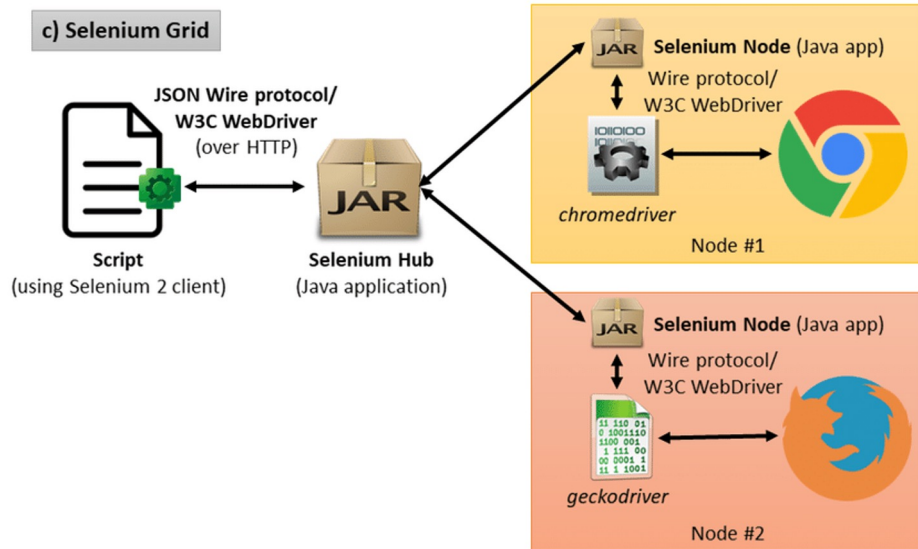


Автоматизация

Selenium – это набор инструментов, которые широко используются для кроссбраузерного тестирования.

```
from selenium import webdriver

options = webdriver.ChromeOptions()
browser =
webdriver.Chrome(options=options)
browser.get("https://codex.so")
browser.implicitly_wait(3)
browser.save_screenshot("page.png")
browser.quit()
```



Расширения браузера

Расширения браузера

WebExtension API — кросс-браузерная система разработки дополнений браузера. Дополнения расширяют и изменяют функциональность браузера. Они разрабатываются с использованием стандартных Веб-технологий – JavaScript, HTML и CSS, а также некоторых специальных JavaScript API, которые позволяют *делать намного больше*, чем то, на что вы способны на любом из сайтов.

- Управлять содержимым сайтов
- Добавлять инструменты для разработки
- Добавлять новые функции в браузер

Минимальное приложение

index.html

```
<h1>WebTech 2024</h1>
```

manifest.json

```
{  
  "manifest_version": 3,  
  "name": "webtech",  
  "version": "1.0.0",  
  "action": {  
    "default_popup": "index.html",  
    "default_title":  
      "WebTech 2024 Browser App"  
  }  
}
```

chrome://extensions/



Расширения



Поиск по расширениям

Загрузить распакованное расширение

Упаковать расширение

Обновить

Все расширения



webtech 1.0.0

Идентификатор: ojcafnggdfolejpagobdbpef...

Сведения

Удалить



Спецификация

Структура manifest.json:

<https://developer.chrome.com/docs/extensions/reference/manifest>

```
"content_scripts": [  
  {  
    "matches": ["https://news.itmo.ru/*"],  
    "js": ["Content.js"]  
  }  
]
```

```
let bobRossImages = [  
  "https://bit.ly/3Ck6DTU", "https://bit.ly/3ozQCVk",  
  "https://bit.ly/3omYDN6", "https://bit.ly/3osrfoi",  
  "https://bit.ly/3qCPjax", "https://bit.ly/3CkRXE6",  
];  
  
const imgs = document.getElementsByTagName("img");  
for (image of imgs) {  
  const index = Math.floor(Math.random() *  
    bobRossImages.length);  
  image.src = bobRossImages[index];  
}
```

Content.js

Наука

Образование

Стартапы и бизнес

Жизнь университета

Медиа

Блог



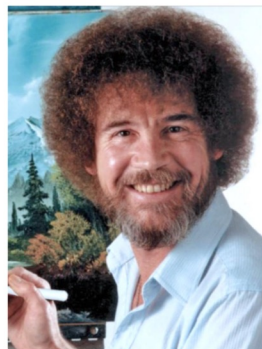
Ректор ИТМО предложил
создать китайско-российский
альянс в области ИИ

22 Апреля 2024



«Возможность, которую
нельзя упускать»:
победители конкурса
ITMO.STARS — о том, как
учиться в университете
бесплатно и заниматься
проектом мечты

18 Апреля 2024



ИТМО уже принимает
документы в бакалавриат и
магистратуру: что нужно
знать, чтобы ничего не
пропустить

11 Апреля 2024

[Далее »](#)

Спецпроект

Выпускники ИТМО — о карьере в науке, бизнесе
и технологиях будущего



Антон Белик

CEO компании Rygobox, выпускник
Университета ИТМО

Разработка выпускника
Университета ИТМО раздast Wi-Fi
и защитит от опасностей в сети

[Читать](#)

Наука



От 3D-печати домов до новых
материалов для
аккумуляторов: лауреат ИТМО



Ученые увидели
топологические эффекты в
новом материальном состоянии



В ИТМО придумали, как
увеличить время жизни
квантовых состояний



Илья Климентьев

“Хотите рассказать о своем проекте,
исследовании или разработке?
Пишите!”

Контакты:

✉ news@itmo.ru

Progressive Web Apps (PWA)

Проблемы

- Хотелось бы, чтобы веб-сайт работал оффлайн (как Google Docs)
- Очень классно, если его можно было бы установить как приложение на смартфоне или ноутбуке
- Писать конечно же только на 1 языке, чтобы не переписывать каждый раз
- Чтобы можно было открыть в браузере
- Чтобы обновляло информацию в фоне без необходимости перезагружать страницу

Electron

<https://www.electronjs.org/>

- Позволяет собрать приложение под разные ОС: windows, linux, macos
- Вместе с исполняемым файлом приложения идёт целый Chromium
- По сути, браузер внутри приложения



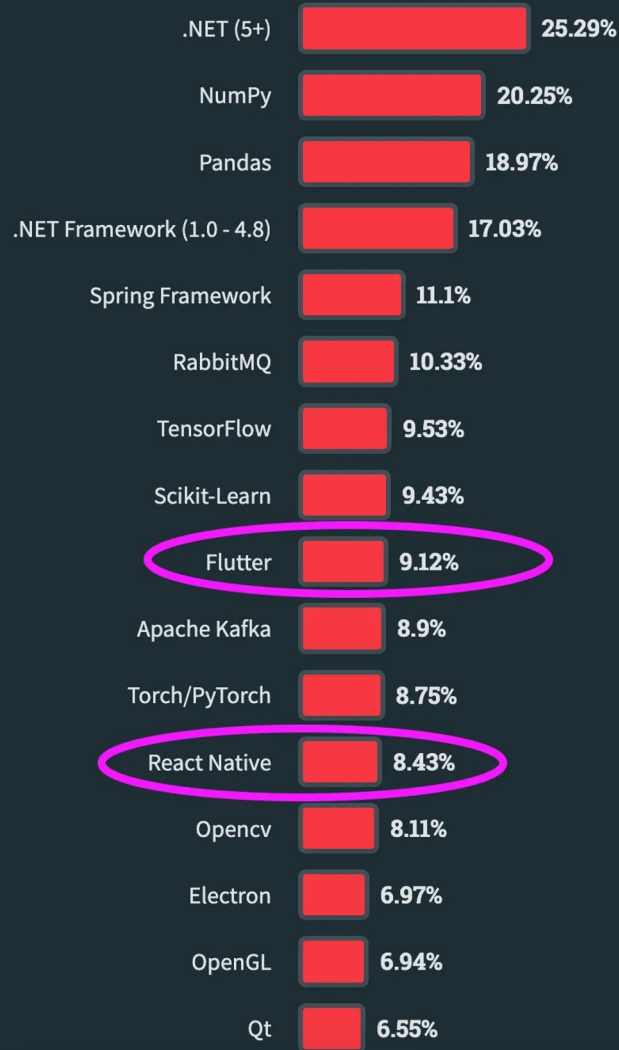
Flutter, React Native

<https://docs.flutter.dev/>

<https://reactnative.dev/>

- Ориентация на UI
- В Flutter используется язык Dart
- У React Native нет официальной поддержки платформ кроме Android и IOS *

<https://microsoft.github.io/react-native-windows/>



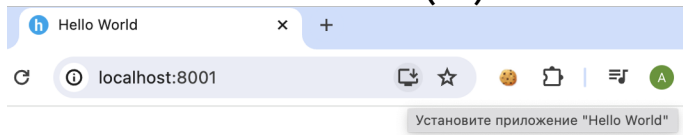
Что такое PWA

Это веб-приложения, написанные с помощью веб-технологий и API, но доступные на любом устройстве.

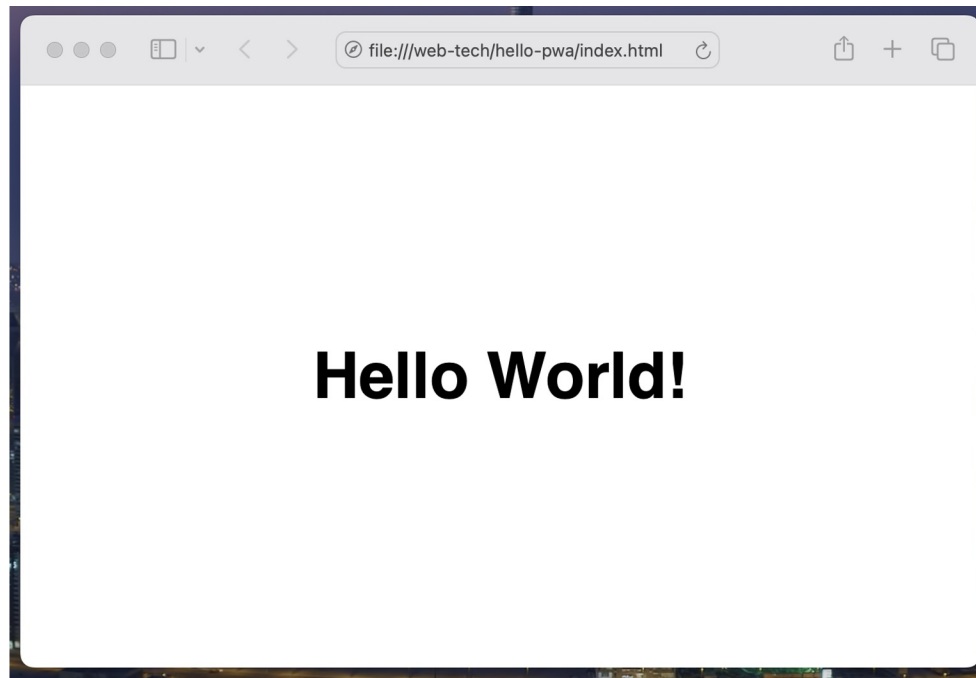
- Устанавливается как приложение на ноутбуке/смартфоне
- Поддерживает Push уведомления
- Можно поделиться просто отправив ссылку (без магазинов приложений)
- Обновление версии через перезагрузку страницы
- Офлайн работа и кеширование

Сразу пример Hello World PWA

- HTML, CSS, JS
- + manifest.json
- + service worker (JS)



Hello World!



Uncaught (in promise) TypeError: Failed to register a ServiceWorker: The URL protocol of the current origin ('null') is not supported. [main.js:6](#)
at window.onload ([main.js:6:15](#))
Access to internal resource at 'file:///manifest.json' from origin 'null' has been blocked by CORS policy: Cross origin requests are only supported for protocol schemes: http, data, isolated-app, chrome-extension, chrome, https, chrome-untrusted. [index.html:1](#)

`python3 -m http.server`

Ограничения

- Проблемы с Apple (вместо PWA – Home Screen Web Apps)
- Не на всех платформах есть полная поддержка некоторых API: Web Bluetooth, WebNFC, ...
- Не все платформы поддерживают PWA (Smart TVs, Smartwatches, Cars, Game consoles)

Манифест

<https://developer.mozilla.org/en-US/docs/Web/Manifest>

```
<link rel="manifest" href="manifest.json" />
```

Обязательные параметры:

- name
- icons
- start_url
- display and/or display_override

Web Workers API

Позволяет выполнять операции в фоновом потоке, отдельном от основного потока выполнения веб-приложения.

- **Dedicated worker** – фоновая задача в виде JS скрипта
- **Shared worker** – фоновый скрипт, к которому можно обращаться из разных контекстов (окно, IFrame и т.д.) в пределах одного Origin
- **Service Worker** – выступает в роли прокси-сервера между веб-приложениями, браузером и сетью. Предназначены для офлайн работы и push-уведомлений.

Dedicated worker

<https://mdn.github.io/dom-examples/web-workers/simple-web-worker/>

```
onmessage = function(e) {  
  console.log('Message received from main script');  
  const result = e.data[0] * e.data[1];  
  if (isNaN(result)) {  
    postMessage('Please write two numbers');  
  } else {  
    const workerResult = 'Result: ' + result;  
    console.log('Posting message back to main');  
    postMessage(workerResult);  
  }  
}
```

```
const myWorker = new Worker("/worker.js");  
const first = document.querySelector("input#number1");  
const second = document.querySelector("input#number2");  
  
first.onchange = () => {  
  myWorker.postMessage([first.value, second.value]);  
  console.log("Message posted to worker");  
};
```

<https://developer.mozilla.org/en-US/docs/Web/API/Worker>

Shared worker

chrome://inspect/#workers

```
onconnect = function (event) {  
    const port = event.ports[0];  
  
    port.onmessage = function (e) {  
        const workerResult = `Result: ${e.data[0]}  
* e.data[1]`;   
        port.postMessage(workerResult);  
    };  
};
```

Worker.js

Две разных страницы в рамках единого Origin:

<https://mdn.github.io/dom-examples/web-workers/simple-shared-worker/>

<https://mdn.github.io/dom-examples/web-workers/simple-shared-worker/index2.html>

Просто добавляем **port**

```
const myWorker = new SharedWorker("worker.js");  
myWorker.port.postMessage([squareNumber.value, squareNumber.value]);  
myWorker.port.onmessage = function (event) { ...
```

Service Worker

Service worker работает в своем контексте: у него нет доступа к DOM, и он работает в потоке, отличном от основного JavaScript.

Жизненный цикл:

1. Download
2. Install
3. Activate

chrome://serviceworker-internals/

Scope: <https://mdn.github.io/dom-examples/service-worker/simple-service-worker/>

Storage key:

Origin: <https://mdn.github.io>

Top level site: <https://mdn.github.io>

Ancestor chain bit: SameSite

Registration ID: 1239

Navigation preload enabled: true

Navigation preload header length: 4

Active worker:

Installation Status: ACTIVATED

Running Status: STOPPED

Fetch handler existence: EXISTS

Fetch handler type: NOT_SKIPPABLE

Script: <https://mdn.github.io/dom-examples/service-worker/simple-service-worker/sw.js>

Version ID: 5027

Renderer process ID: 0

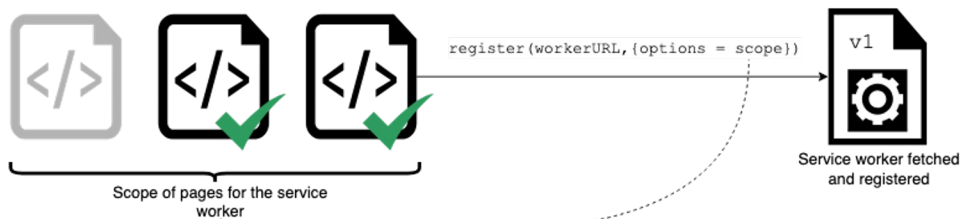
Renderer thread ID: -1

DevTools agent route ID: -2

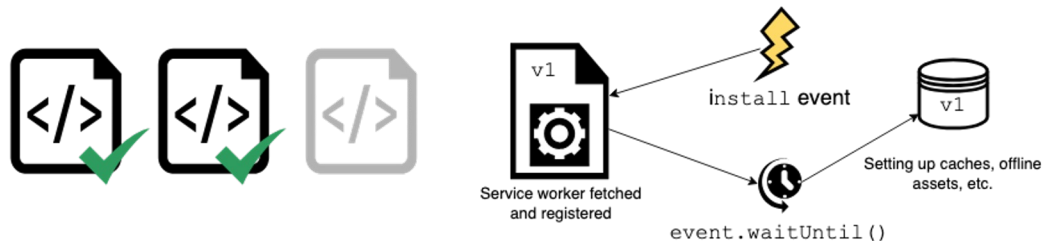
0. Initial situation, no service worker



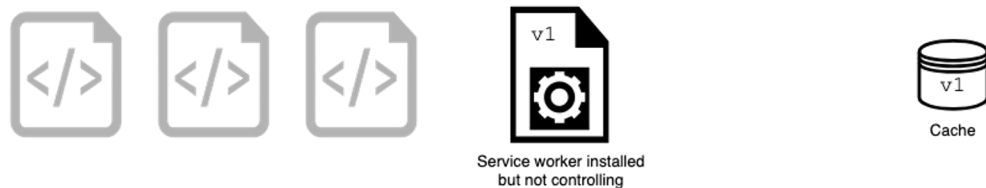
1. Registration of the first version of a service worker



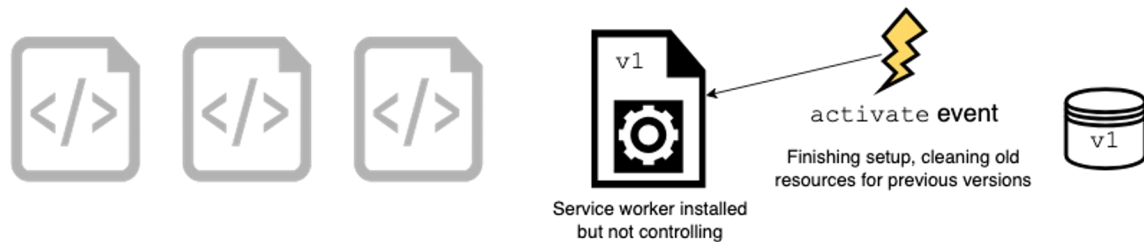
2. Installation



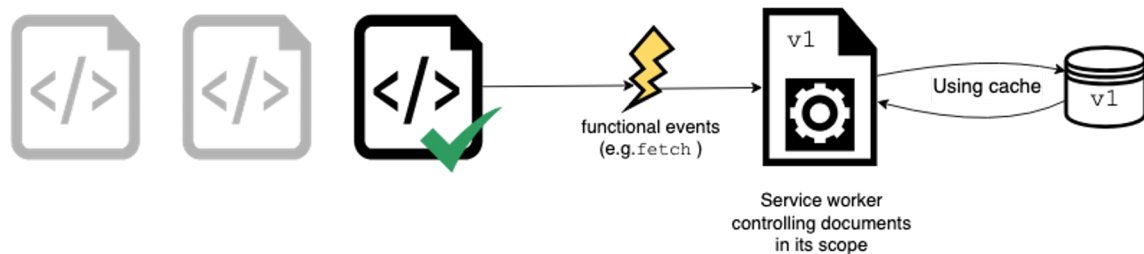
3. Waiting for clients to be closed



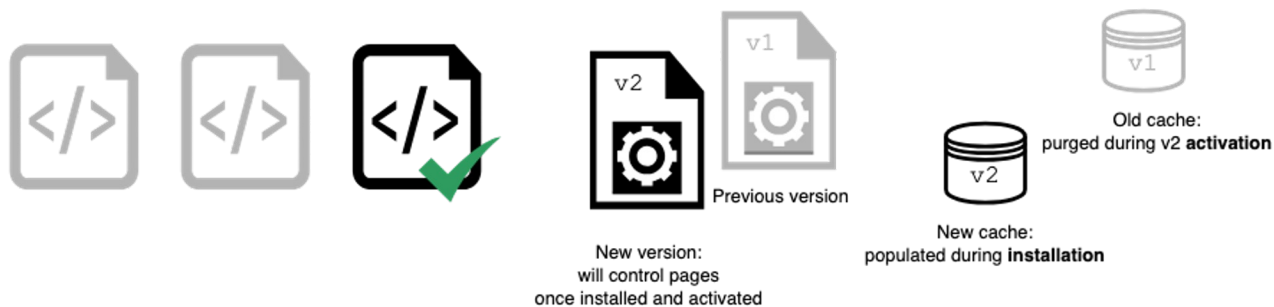
4. Activation



5. Activated



6. Replacement



Подключаем Service Worker'a

```
navigator.serviceWorker.register('sw.js')  
  .then(reg => console.log('SW registered!', reg))  
  .catch(err => console.log('Error!', err));
```

```
self.addEventListener('install', event => {  
  console.log('installing');  
})  
  
self.addEventListener('activate', event => {  
  clients.claim();  
  console.log('activated');  
});
```

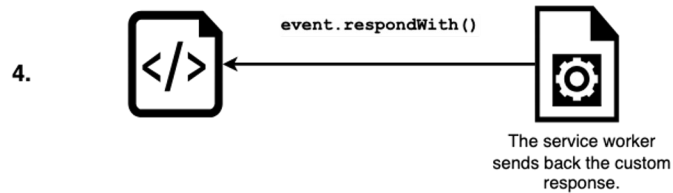
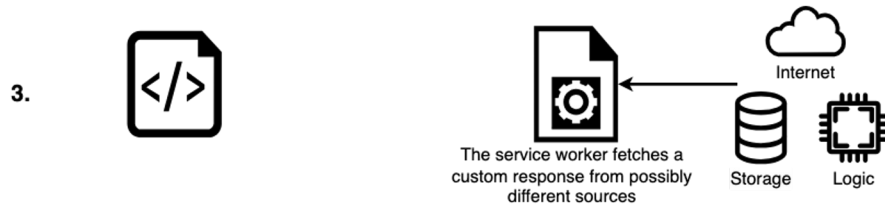
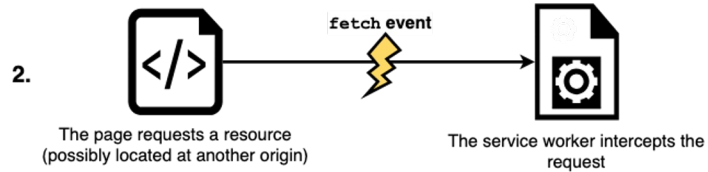
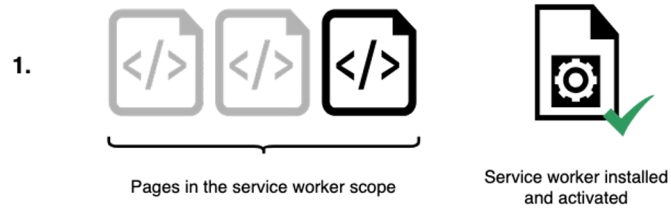
sw.js

Cache

<https://developer.mozilla.org/en-US/docs/Web/API/Cache>

Доступен Service Worker'y через API

- `cache.add(new Request('/data.json'));`
- `cache.put('/test.json', new Response('{ "foo": "bar" }'));`
- `cache.match(request);`



Пример использования Cache API и Cats API

http://localhost:8000/cat/0

GET

● 200 OK (from service worker)

strict-origin-when-cross-origin

```
function getCat() {  
  
    const img = new Image();  
  
    img.width = 300;  
  
    img.src = `/cat/${cats}`;  
  
    img.style.display = 'block';  
  
    document.getElementById("gallery").appendChild(img);  
  
    cats += 1;  
}
```

Shared storage

Cache storage

offline - http://localhost:8000/

cats-storage - http://localhost:8000/

Background services

Back/forward cache

Background fetch

Background sync

Bounce tracking mitigations

Notifications

Payment handler

Periodic background sync

Speculative loads

Push messaging

Reporting API

Frames

top

#	Name	Response-Type	Content-Type	Content-Length	Time Cached	V
0	/cat/0		cors image/jpeg	16,577	27.04.2024, 18:35:29	
1	/cat/1		cors image/jpeg	49,882	27.04.2024, 18:35:29	
2	/cat/10		cors image/jpeg	67,314	27.04.2024, 18:35:39	
3	/cat/11		cors image/jpeg	52,224	27.04.2024, 18:35:40	
4	/cat/12		cors image/jpeg	41,163	27.04.2024, 18:35:40	
5	/cat/13		cors image/jpeg	52,223	27.04.2024, 18:35:41	
6	/cat/2		cors image/jpeg	85,576	27.04.2024, 18:35:29	
7	/cat/3		cors image/jpeg	153,385	27.04.2024, 18:35:29	
8	/cat/4		cors image/jpeg	62,680	27.04.2024, 18:35:29	
9	/cat/5		cors image/jpeg	276,738	27.04.2024, 18:35:34	
10	/cat/6		cors image/jpeg	85,548	27.04.2024, 18:35:35	

Headers

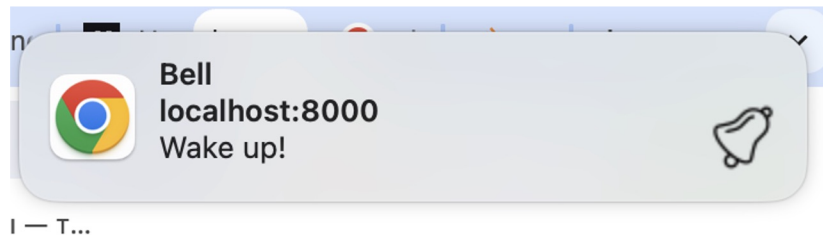
Preview

Уведомления

https://developer.mozilla.org/en-US/docs/Web/API/Notifications_API

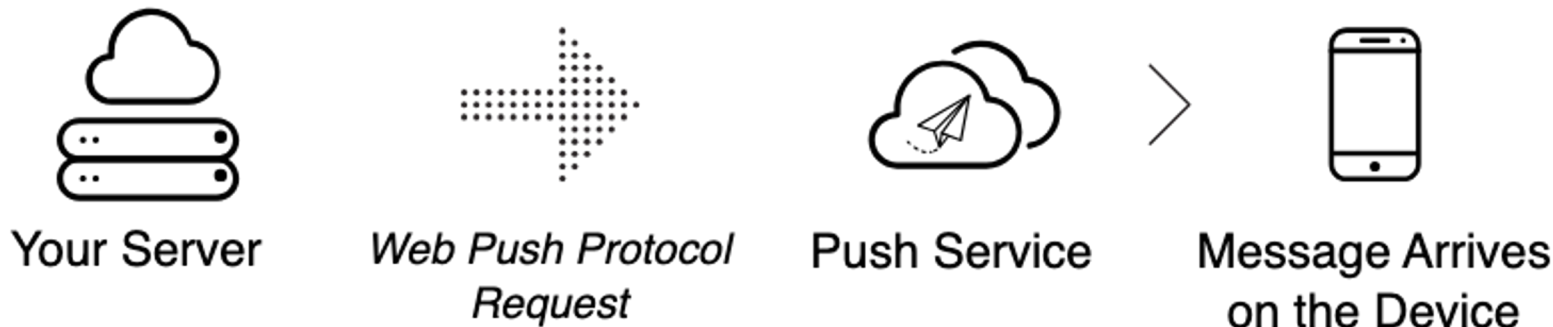
```
Notification.requestPermission().then((result) => {  
    document.getElementsByTagName("textarea")[0].innerHTML = result;  
});
```

```
const img = "bell.png";  
    const text = `Wake up!`;  
    const notification = new  
Notification("Bell", { body: text, icon:  
img });
```



Push API

Push API предоставляет веб-приложениям возможность принимать сообщения с сервера независимо от того, запущено веб-приложение прямо сейчас или нет.



Web Push сервис

Push-сервис — сторонняя служба (например, Google FCM), которая получает запросы на отправку push-уведомлений, проверяет их и доставляет уведомления в соответствующий браузер.

<https://firebase.google.com/docs/cloud-messaging?hl=ru>



*1. Message Arrives
on Device*



*2. Browser Wakes
Up Service Worker*

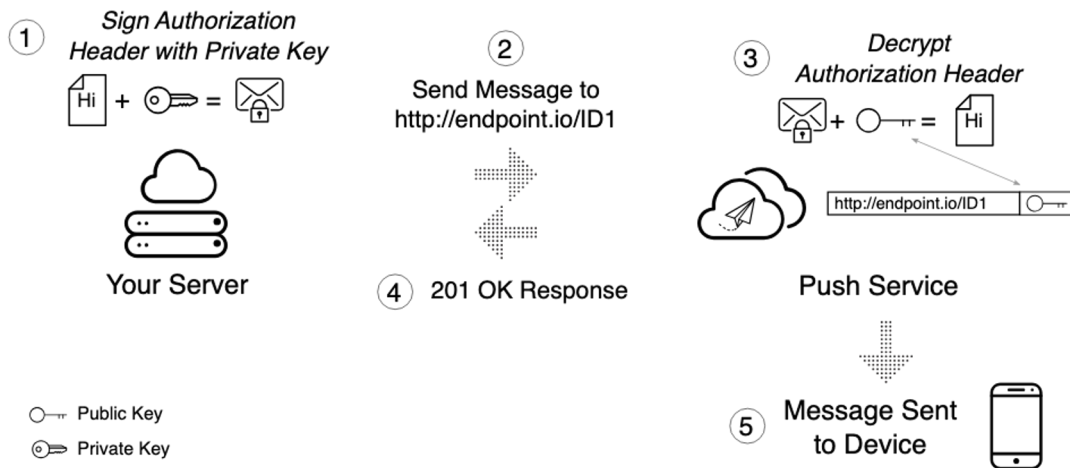


*3. Push Event
is Dispatched*

Web Push протокол

<https://web.dev/articles/push-notifications-web-push-protocol>

- Устройство стало доступно
- TTL сообщения истёк.



Пример

Service worker подписывается на событие и ждёт Event **push**

```
const subscription = await self.registration.pushManager.subscribe({
  userVisibleOnly: true,
  applicationServerKey: urlBase64ToUint8Array("<...>")
})

self.addEventListener("push", e => {
  self.registration.showNotification("Wohoo!!", { body: e.data.text() })
})
```

На сервере подключаемся к push сервису и позже отправляем сообщение

```
webpush.setVapidDetails(
  'mailto:YOUR_MAILTO_STRING',
  apiKeys.publicKey,
  apiKeys.privateKey
)

webpush.sendNotification(subDatabase[0], "Hello world");
```

Итоги

1. Рассмотрели несколько дополнительных браузерных API
2. Научились использовать WebDriver и Selenium
3. Разобрались, как работают расширения браузера и как их разрабатывать
4. Изучили тему прогрессивных Веб приложений
5. Познакомились с принципами работы Service Workers, Web Push API и Notification API

Полезные ссылки

- Статистика популярных фреймворков – <https://survey.stackoverflow.co/2023/#other-frameworks-and-libraries>
- PWA Hello World гайд – <https://medium.com/james-johnson/a-simple-progressive-web-app-tutorial-f9708e5f2605>
- Серия статей про Service Worker'ы – <https://habr.com/ru/companies/ruvds/articles/350486/>
- Web API – <https://developer.mozilla.org/en-US/docs/Web/API>
- Web Driver Spec – <https://w3c.github.io/webdriver/#endpoints>
- Chrome driver – <https://chromedriver.chromium.org/downloads>
- Firefox driver – <https://github.com/mozilla/geckodriver/releases>
- Python и selenium – <https://www.browserstack.com/guide/take-screenshot-with-selenium-python>
- PWA демки – <https://developer.chrome.com/docs/capabilities/fugu-showcase>
- Web Push firebase – <https://firebase.google.com/docs/cloud-messaging?hl=ru>
- Chrome extension manifest – <https://developer.chrome.com/docs/extensions/reference/manifest>
- Web Push Protocol – <https://web.dev/articles/push-notifications-web-push-protocol>

Вопросы?